

UNIVERSIDAD DE CARABOBO
Facultad Experimental de Ciencias y Tecnología (FacyT)
Bárbula - Carabobo

Informe
Práctica de Laboratorio 1

Bachilleres:

Samuel Cordero	CI: 31.678.592
Dervis Martínez	CI: 31.456.326

Docente: José Canache

Julio, 2025

INFORME DE LABORATORIO 2 - ARQUITECTURA MIPS32

Algoritmos de Ordenamiento: Bubble Sort y QuickSort

1. Diferencias entre registros temporales y guardados

Los registros temporales (\$t0–\$t9) son utilizados para almacenar valores intermedios que no requieren persistencia después de llamadas a procedimientos. Su contenido puede sobrescribirse sin previo aviso. Los registros guardados (\$s0–\$s7), por otro lado, deben preservar su valor entre llamadas a funciones, y si se usan dentro de un procedimiento, deben guardarse y restaurarse usando la pila.

En esta práctica, Bubble Sort fue implementado completamente dentro del procedimiento principal (`main`), por lo que no se utilizaron subrutinas y únicamente se emplearon registros temporales. Sin embargo, QuickSort, al ser un algoritmo recursivo, sí requiere dividir el problema en subproblemas mediante llamadas a funciones (subrutinas). En su implementación en MIPS32, esto implica que es obligatorio el uso del registro \$ra para retornar correctamente a la llamada anterior y el uso de registros guardados o pila para mantener el estado entre llamadas, lo cual complica pero también enriquece la estructura del algoritmo.

2. Diferencias entre \$a0–\$a3, \$v0–\$v1 y \$ra

Los registros \$a0–\$a3 se utilizan para pasar argumentos a funciones o al sistema. Los registros \$v0 y \$v1 se usan para retornar valores o para definir servicios en llamadas `syscall`.

\$ra (return address) guarda la dirección de retorno tras una llamada a subrutina con `jal`. En esta práctica, \$a0 fue utilizado para imprimir datos, y \$v0 para invocar servicios del sistema como imprimir enteros o finalizar el programa. \$ra no se utilizó ya que Bubble Sort fue implementado directamente dentro del `main`, sin subrutinas. En una versión modular o en QuickSort (que puede implementarse recursivamente), el uso de \$ra sería esencial para manejar los retornos correctamente.

3. Impacto del uso de registros frente a memoria

El acceso a registros es considerablemente más rápido que a memoria. En Bubble Sort, los registros se usaron para índices, comparaciones y banderas, reduciendo los accesos a memoria. En cada iteración, los valores se cargaban, manipulaban en registros, y solo se escribían de vuelta si se necesitaba un intercambio. Esto reduce el uso de instrucciones costosas como `lw` y `sw`.

Si el algoritmo operara directamente sobre memoria sin usar registros, aumentaría la latencia y se ralentizaría la ejecución. En MIPS32, los registros ejecutan en un ciclo, mientras que la memoria puede tardar varios.

En QuickSort, se usan más accesos a memoria debido a la necesidad de pila para recursividad, pero se compensa con menos comparaciones, logrando mejor rendimiento general que Bubble Sort en arreglos grandes.

4. Estructuras de control y eficiencia

Bubble Sort usa bucles anidados que repiten muchas comparaciones. QuickSort usa división recursiva del arreglo, lo que disminuye el número de comparaciones promedio. MIPS exige uso explícito de instrucciones de control como `beq`, `bne`, `j`, haciendo el manejo de control más delicado pero transparente.

El uso de estructuras de control bien diseñadas permite una ejecución predecible y efectiva. En cambio, estructuras mal definidas o bucles infinitos impactan negativamente en el rendimiento y dificultan la depuración. En MIPS, cada instrucción de control se traduce directamente en ciclos de ejecución, por lo que estructuras innecesarias o redundantes penalizan el tiempo total de procesamiento.

5. Complejidad computacional

- **Bubble Sort:** Mejor caso: $O(n)$; Peor y promedio caso: $O(n^2)$
- **QuickSort:** Promedio: $O(n \log n)$; Peor caso: $O(n^2)$ (mal pivote)

QuickSort es más eficiente en la mayoría de los casos. Además, en un entorno MIPS32, implementar QuickSort implica trabajar con la pila manualmente para manejar las llamadas recursivas, lo que agrega complejidad en la administración de memoria y control del flujo. Bubble Sort, por ser iterativo, es más fácil de implementar, pero su complejidad cuadrática lo vuelve inviable para grandes volúmenes de datos. La elección del algoritmo tiene implicaciones directas tanto en el rendimiento como en la estructura del código ensamblador.

6. Fases del ciclo de ejecución en MIPS32

1. **Fetch:** Se obtiene la instrucción desde memoria.
2. **Decode:** Se decodifica y se leen registros fuente.
3. **Execute:** Se realiza la operación en la ALU.
4. **Memory:** Acceso a memoria si aplica.
5. **Write Back:** Se escribe el resultado al registro destino.

7. Tipos de instrucciones predominantes

- **Tipo R:** `add`, `sub`, `mul` para operaciones con registros.
- **Tipo I:** `lw`, `sw`, `beq`, `bne` para memoria y control.
- **Tipo J:** `j` para saltos incondicionales.

Estas instrucciones fueron las más usadas porque permiten un control total sobre el flujo del programa, acceso a memoria y cálculo de operaciones. Los algoritmos de ordenamiento requieren operaciones constantes sobre elementos de un arreglo y toma de decisiones, por lo que las instrucciones R e I son fundamentales.

8. Efecto del abuso de instrucciones de salto

Usar excesivamente saltos (incondicionales o condicionales) puede llevar a una lógica confusa y menos eficiente. En especial si no se estructuran adecuadamente, pueden generar bucles infinitos o errores lógicos difíciles de depurar.

Desde el punto de vista del rendimiento, abusar de saltos puede causar que el flujo de ejecución se vuelva fragmentado y propenso a errores de predicción, lo cual afecta negativamente al rendimiento del pipeline de ejecución en arquitecturas RISC como MIPS. En contraste, el uso de estructuras lineales bien definidas permite un mejor aprovechamiento del hardware y facilita la lectura y depuración del código.

9. Ventajas del modelo RISC de MIPS

- Instrucciones simples y uniformes.
- Ejecución eficiente por ciclo.
- Facilidad para predecir rendimiento.
- Menor complejidad en el diseño de hardware.
- Mayor portabilidad en simuladores educativos.

10. Uso de Step/Step Into en MARS

Estas herramientas permitieron la depuración detallada del código, observando línea por línea cómo cambian los valores de registros y memoria. Esto fue especialmente útil para verificar el comportamiento de condiciones, intercambios, límites de bucles y asegurar que los valores no se sobrescribieran indebidamente.

En el caso de QuickSort, fue clave para asegurar que la recursión funcionara correctamente y no se saliera del arreglo.

11. Herramientas más útiles en MARS

- **Register Panel:** observación de registros en tiempo real.
- **Data Segment:** ver el arreglo en memoria.
- **Instruction Step:** control fino del flujo de ejecución.

12. Visualización del camino de datos

Tipo R (add): El procesador accede a dos registros fuente, los envía a la ALU, realiza la operación y almacena el resultado en un registro destino. Esto permite visualizar de forma clara el flujo de datos dentro del procesador sin intervención de memoria.

Tipo I (lw): El procesador toma un registro base y un offset inmediato, los suma para obtener una dirección efectiva de memoria, accede a esa posición y transfiere el dato al registro destino. Aquí se visualiza el camino de datos desde la memoria hacia el procesador.

13. Justificación del algoritmo alternativo

QuickSort fue elegido por su mejor rendimiento promedio ($O(n \log n)$) y su capacidad de manejar grandes volúmenes de datos eficientemente. Implementarlo en MIPS exige control manual de pila, segmentación y condiciones de partición, fortaleciendo habilidades en bajo nivel.

14. Análisis y discusión de resultados

La comparación entre Bubble Sort y QuickSort evidenció diferencias importantes en rendimiento y complejidad. Bubble Sort es sencillo y directo, pero su eficiencia se degrada rápidamente a medida que crece el tamaño del arreglo. Por otro lado, QuickSort, aunque más complejo de implementar en MIPS debido al uso de pila y subrutinas, ofreció mejoras claras en número de comparaciones y tiempo de ejecución.

Durante la implementación, se observaron retos particulares en la gestión de direcciones, límites de segmentos y el control correcto del flujo. QuickSort, al requerir más planificación en la gestión de memoria, permitió aplicar conceptos avanzados como recursión simulada y preservación de contexto.

Conclusión

Esta práctica consolidó conocimientos fundamentales sobre la arquitectura MIPS32 y su aplicación a problemas algorítmicos. La comparación entre algoritmos evidenció la importancia de elegir bien según el contexto. MARS permitió una visualización clara y un entorno adecuado para la depuración. Además, reforzó el valor del diseño algorítmico estructurado como base para eficiencia y control en arquitecturas de bajo nivel.